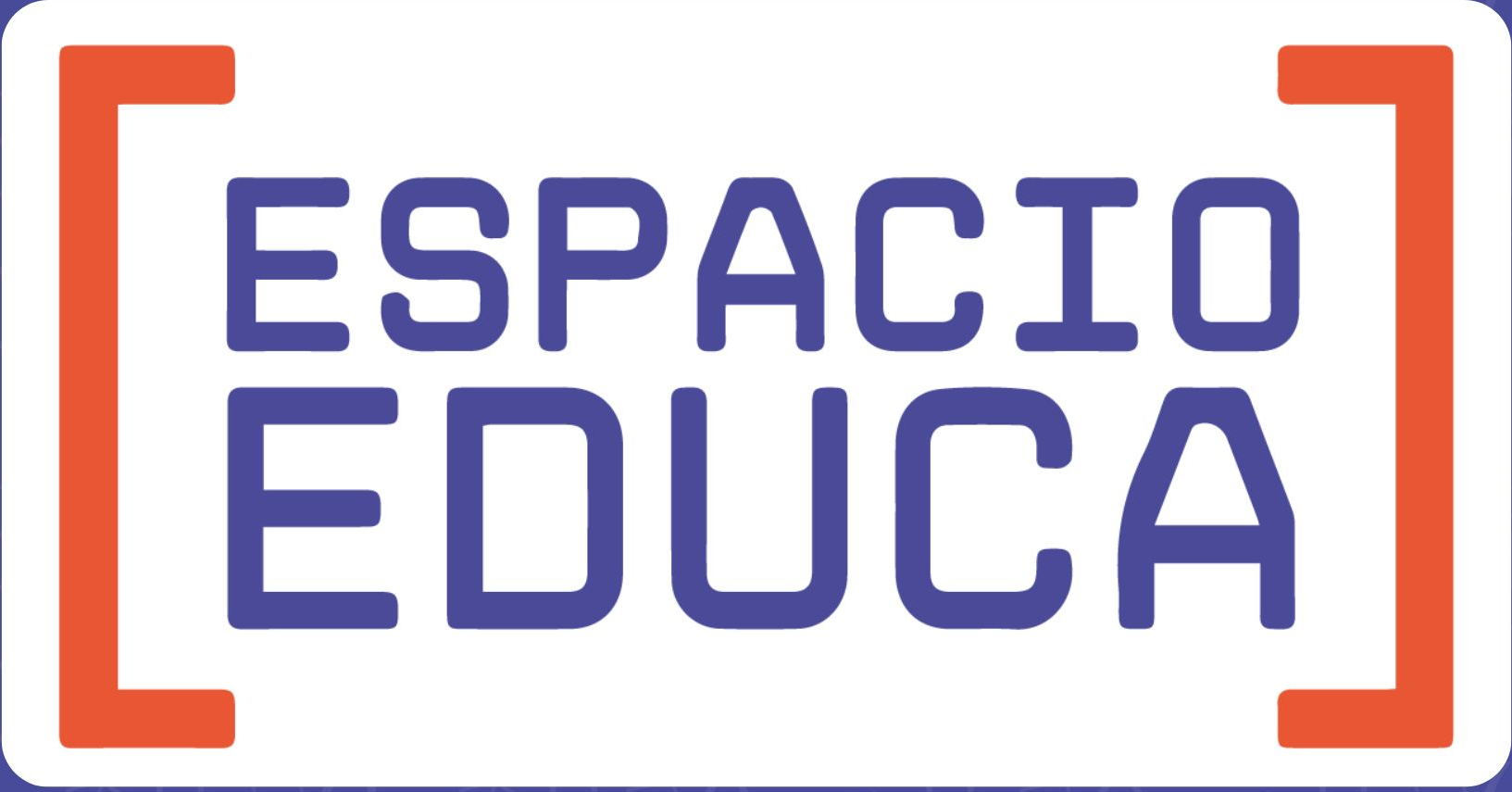




**ESPACIO
EDUCA**

[HOY VAMOS A APRENDER A...]

Aplicar la herencia de clases para hacer que nuestro código sea reusable y mantenible.

Vocabulario: herencia, reusable, mantenible, superclase, subclase, extender

HAZLO AHORA



Repasemos

¿Qué atributos o características podemos usar para describir la taza?

¿Qué métodos podemos utilizar para describir el comportamiento o hacer cálculos sobre la taza?



DISCUSIÓN



¿A qué les
recuerda la
palabra
herencia?



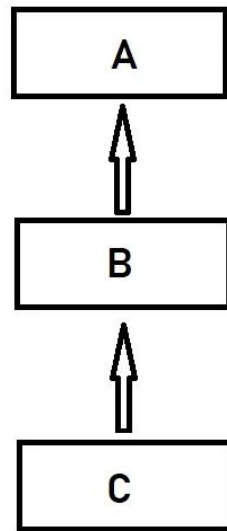


Herencia en POO

La Programación Orientada a Objetos como paradigma, se basa en cuatro pilares fundamentales:

- Abstracción
- Encapsulamiento
- Polimorfismo
- **Herencia**

Este principio tiene como finalidad hacer que el código sea **mantenible** y **reutilizable** permitiendo crear una jerarquía de clases.



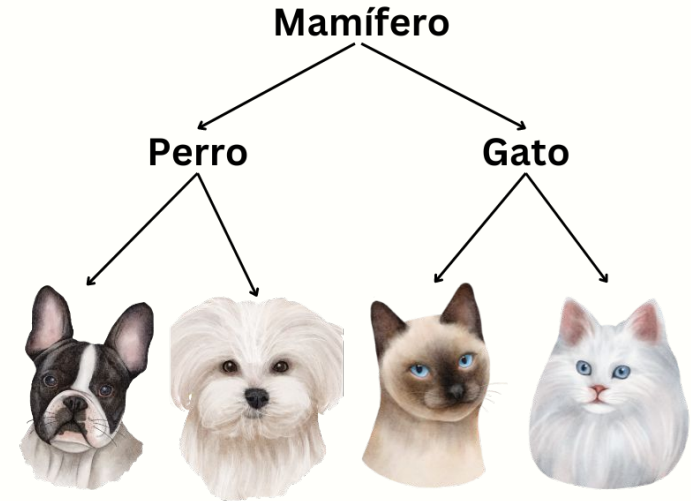


Herencia en POO

En oportunidades nos encontramos con objetos que comparten ciertas características o atributos.

La herencia de clases nos permite basar una clase de otra cuya **implementación** sea **similar**.

Además de describir el comportamiento heredado, las clases hijas pueden describir su **propio comportamiento**.

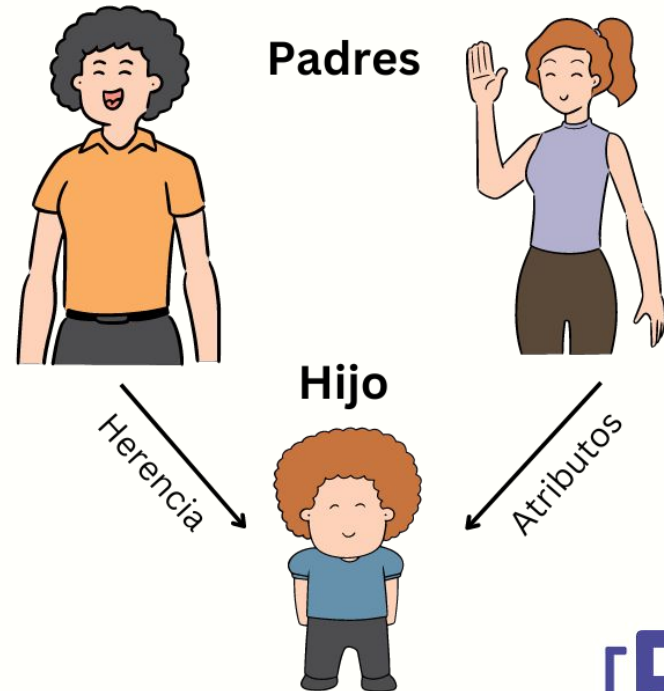




Herencia en POO

Se conoce a la clase que se hereda de una **clase padre** (superclase) como **clase hija** (subclase).

Los objetos de esa clase poseen los atributos y comportamientos heredados.



PREGUNTA FLASH



Identifica la subclase y la superclase en esta imagen.

```
class Perro extends Mamifero {  
  constructor(nombre, altura, peso) {  
    // Inicializa la clase con el constructor de la superclase  
    super(nombre, altura, peso);  
  }  
  ladrar() {  
    console.log('Woof woof!!');  
  }  
}
```





Herencia en POO

```
class Mamifero {  
  constructor (nombre, altura, peso){  
    this.nombre = nombre;  
    this.altura = altura;  
    this.peso = peso;  
  }  
  sonido() {  
    console.log('Sonido...');  
  }  
}
```

Este es el **poder de la herencia**.

Las clases hijas pueden tener **atributos y métodos adicionales** e incluso **modificar** los descritos en la clase padre.

Usamos la palabra reservada **extends** para indicar que la clase *Perro* herede de la clase *Mamífero*.

```
class Perro extends Mamifero {  
  constructor(nombre, altura, peso) {  
    // Inicializa la clase con el constructor de la superclase  
    super(nombre, altura, peso);  
  }  
  ladrar() {  
    console.log('Woof woof!!');  
  }  
}
```





¿Cómo **agregar** atributos y métodos en las subclases?

```
class Mamifero {  
  constructor (nombre, altura, peso){  
    this.nombre = nombre;  
    this.altura = altura;  
    this.peso = peso;  
  }  
  sonido() {  
    console.log('Sonido...');  
  }  
}
```

Declaramos los atributos que se deseen agregar

Definimos e implementamos los métodos que se deseen agregar

```
class Perro extends Mamifero {  
  constructor(nombre, altura, peso, frecuenciaLadridos) {  
    super(nombre, altura, peso);  
    this.frecuenciaLadridos = frecuenciaLadridos;  
  }  
  ladrar() {  
    console.log('Woof woof!!');  
  }  
}
```





¿Cómo
sobreescribir
métodos ya
existentes en las
subclases?

```
public class Mamifero{  
    private double altura;  
    private double peso;  
  
    public void sonido(){  
        System.out.println("Sonido...");  
    }  
}
```

El método sonido() es
un método ya existente
en la superclase
Mamífero.

Para definir un comportamiento diferente del mismo en la subclase *Perro*, debemos implementar el método nuevamente, esto se conoce como **sobreescritura de métodos**.

```
class Perro extends Mamifero {  
    constructor(nombre, altura, peso, frecuenciaLadridos) {  
        super(nombre, altura, peso);  
        this.frecuenciaLadridos = frecuenciaLadridos;  
    }  
  
    sonido() {  
        this.ladRAR();  
    }  
  
    ladRAR() {  
        console.log('Woof woof!!');  
    }  
}
```

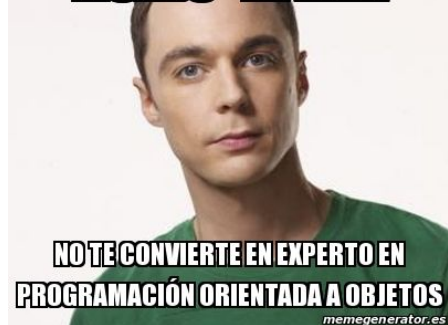


PREGUNTA FLASH



¿A qué hace referencia **super**?

SABER HACER EL "HOLA MUNDO" EN JAVA



```
class Perro extends Mamifero {  
    constructor(nombre, altura, peso, frecuenciaLadridos) {  
        super(nombre, altura, peso);  
        this.frecuenciaLadridos = frecuenciaLadridos;  
    }  
  
    sonido() {  
        super.sonido();  
        this.ladrar();  
    }  
}
```





¿A qué hace referencia **super**?

```
class Perro extends Mamifero {  
    constructor(nombre, altura, peso, frecuenciaLadridos) {  
        super(nombre, altura, peso);  
        this.frecuenciaLadridos = frecuenciaLadridos;  
    }  
  
    sonido() {  
        super.sonido();  
        this.ladrar();  
    }  
}
```

“Super” hace referencia a la superclase. En este caso, vemos que se invoca el método sonido de la superclase *Mamífero*.





3:00

¡Ahora es tu turno!

Escribe en una hoja o en tu cuaderno un ejemplo de 3 clases que sean similares. Identifica qué atributos podrían compartir y qué comportamientos tienen en común. ¿Cómo podrías aplicar la herencia para esas clases? ¿Cuál sería la superclase?



DISCUSIÓN



¿Qué
ventajas y
desventajas
ven de usar la
herencia de
clases?

